

Q1

编写一个void createAndWait(int N)函数，这个函数创建N个线程来执行work函数，每个线程的标识符从0到N-1，然后等待所有线程执行完毕。目标是最大化并发。

```
// 定义 work 函数，它接受一个整数参数 index 作为线程的标识符
void work(int index) {
    // 打印开始信息和线程标识符
    std::cout << "started " << index << std::endl;
    // 生成一个0到1000之间的随机数，用作休眠时间
    auto r = std::rand() % 1000; // 0 到 1 秒
    // 使线程休眠r毫秒
    std::this_thread::sleep_for(std::chrono::milliseconds(r));
    // 打印完成信息和线程标识符
    std::cout << "finished " << index << std::endl;
}
```

模拟一个可能会在不同线程上并发运行的任务。它首先打印出一个标识符，然后休眠一个随机的时间段，模仿任务执行的不确定性和耗时，最后宣告任务完成。main函数负责接收线程数量的参数，并启动整个多线程过程

```
// 定义函数 createAndWait，接受一个整数 N 作为参数，表示要创建的线程数量
int createAndWait(int N) {
    // 定义一个 thread 类型的向量，用于存储线程
    vector<thread> threads;
    // 为向量预留 N 个元素的空间，以提高性能
    threads.reserve(N);
    // 循环 N 次，每次创建一个新线程
    for (int i=0; i < N; i++) {
        // 使用 emplace_back 来直接在向量中构造 thread 对象
        // emplace_back 比 push_back 更高效，因为它避免了额外的拷贝或移动操作
        threads.emplace_back(work, i);
        // 打印出线程被创建的消息和线程的索引
        std::cout << "created " << i << std::endl;
    }
    // 循环遍历所有线程，使用 join 等待每个线程完成
    for (int i=0; i < N; i++) {
        // join 函数会阻塞当前线程，直到对应的线程执行完毕
        threads[i].join();
        // 打印出线程已经完成的的消息和线程的索引
        std::cout << "joined " << i << std::endl;
    }
    // 函数返回 0
    return 0;
}
```

有一个主线程（main）和三个被创建的子线程（T0、T1、T2）。每个子线程执行三条指令：打印其标识符（"mon id est ..."），执行随机时间的休眠（"sleep(rand)"），以及在完成后打印结束信息（"id XXX se finit"）。主线程首先创建这些子线程，然后通过join方法等待它们各自完成执行。

这里讨论的“部分顺序”（ordre partiel）指的是，在多线程程序中，虽然我们不知道主线程将按顺序创建T0、T1和T2（即T0的创建在M1之后，T1的创建在M2之后），但我们无法确定T0和T1的启动（T0.1和T1.1）哪一个会先发生。换句话说，子线程T0和T1的启动顺序在程序执行时是不确定的，它们都会在它们各自的创建之后开始执行，但具体哪个线程先开始执行则由操作系统的线程调度策略和其他运行时条件决定

```
int main(int argc, const char **argv) {
    // 默认线程数量为
    int N = 3;
    // 如果提供了命令行参数，将第一个参数转换为整数作为线程数量
    if (argc > 1) {
        N = atoi(argv[1]);
    }
    // 设置随机数种子，以确保每次程序运行的随机性
    std::srand(std::time(nullptr));
    // 调用createAndWait函数，传入线程数量N
    return pr::createAndWait(N);
}
```

Q2

les entrelacements possibles

如果执行一个多线程程序，会有哪些可能的线程交错执行（entrelacements）情况

每个线程会按顺序执行其指令，除非硬件或编译器介入进行了乱序执行，但在这个讨论阶段我们不考虑这个因素。

父线程（main函数）创建子线程的操作会在这些子线程开始执行任何指令之前完成。

子线程完成其执行的操作会在父线程的join调用之前完成。

其他的指令没有可比性，或者说它们的执行顺序是不确定的

chaque thread exécute ses instructions dans l'ordre (sauf si le matériel/compilo s'en mêle avec out of order execution. Mais n'en parlons pas à ce stade.)

- les créations par le père précèdent le début des exécutions des ls
- les terminaisons par les ls précèdent le join du père
- les autres instructions ne sont pas comparables/ordonnées

Le dessin suivant exhibe ça :

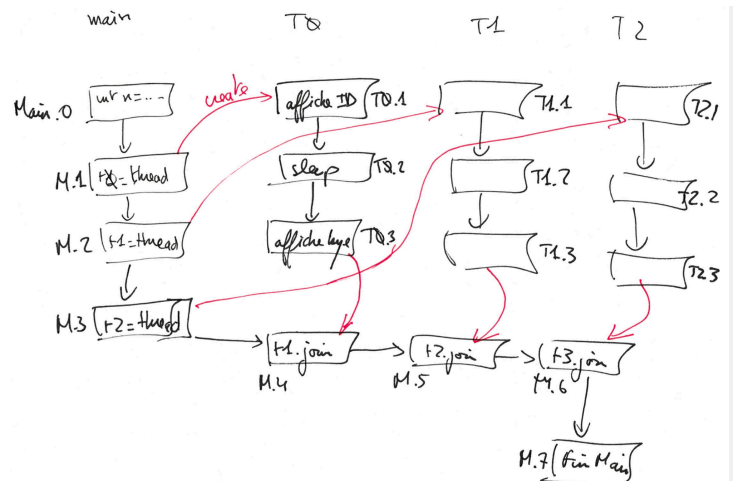
- en noir les actions liées au sein du code d'un thread
- en rouge les synchros inter-thread

On dessine ici pour trois thread créés, chacun ayant trois instructions : mon id est ... , sleep(rand), id XXX se nit

Le main crée puis join les thread.

On remarque un ordre partiel : il est impossible de déterminer si T0.1 précède T1.1 par exemple. Les deux actions seront respectivement après M1 et après M2 (deux actions qui sont ordonnées au sein du main) mais leur ordre n'est pas fixé par le programme en l'état.

Ce dessin permet de capturer tous les entrelacements possibles si on le lit correctement.



Q3 如果在线程创建后、调用join方法前，添加了一个打印"created id"的操作，我们怎么确定消息"created"/"started"/"finished"/"join"之间的顺序关系？

回答：

"started"在"finished"之前（这是在子线程内部顺序执行的）。

"created"在"joined"之前（这是在父线程中顺序执行的）。

"finished"在"joined"之前（因为join的语义意味着同步，即等待子线程完成）。

但是"created"与"started"和"finished"之间是不可比较的，也就是说我们不能确定"created"消息是在"started"还是"finished"消息之前打印的。具体来说，我们可能观察到"created"在"started"和"finished"之间的任意位置。

- started précède finished (ordonné dans fils).
 - created précède joined (ordonné dans pere).
 - nished précède joined (sémantique join = synchro)
- MAIS created est incomparable à started et nished, on peut observer c,s,f ou s,c,f ou s,f,c.
En particulier on peut voir passer nished avant created !

Q4 创建多个线程，每个线程都执行相同的函数jackpot，该函数对一个共享的Compte实例进行操作。

```
#include <vector> // 包含标准库vector
#include <thread> // 包含线程相关功能
using namespace std; // 使用标准命名空间

// 定义一个名为Compte的类
class Compte {
    int solde; // 类的私有变量，用来存储账户余额

public:
    // 构造函数，初始化账户余额，默认为0
    Compte(int solde=0):solde(solde) {}

    // 成员函数，用来向账户中添加金额
    void crediter (size_t val) { solde+=val; }

    // 常量成员函数，返回当前账户的余额
    int getSolde() const {return solde;}
};

// 定义常量JP，表示jackpot函数中要添加到账户的金额
const int JP = 10000;
// 定义jackpot函数，它接受一个Compte对象的引用
void jackpot(Compte & c) {
    // 循环JP次，每次调用crediter函数向账户中添加1单位的金额
    for (int i=0; i < JP; i++)
        c.crediter(1);
}

// 定义常量NB_THREAD，表示将要创建的线程数量
const int NB_THREAD = 10;
```

```
int main () {
    vector<thread> threads; // 创建一个thread对象的容器
    threads.reserve(NB_THREAD); // 预先分配空间以存储线程，避免重新分配

    Compte c; // 实例化Compte对象
    for (int i=0; i < NB_THREAD; i++) {
        // 创建线程，使用std::ref确保传递给线程的是Compte对象的引用
        threads.emplace_back(jackpot, std::ref(c));
    }
    for (auto & t : threads) {
        t.join(); // 等待所有线程完成
    }

    // 检查Compte对象的余额是否等于预期值（每个线程都应该增加JP到余额）
    if (c.getSolde() == N*JP) {
        cout << "Dragon dort sur trésor " << c.getSolde() << endl;
        return 0;
    } else {
        cout << "Ma cassette, mon sang !" << c.getSolde() << endl;
        return 1;
    }
}
```

Q5

因为没有适当的同步机制来控制线程间的操作顺序，结果是不确定的。这是因为多个线程可能会在不同的时刻读取和修改共享变量，导致一些增加到余额的操作被覆盖或丢失。

没有同步机制的情况 ("Sans atomic") :

这种情况下，每个线程读取余额 (load)、增加余额 (add)、存储新余额 (store) 的操作是分开的，可能会相互交错进行。例如，如果两个线程几乎同时读取了相同的余额，它们各自增加1后存储，最终只有1的增加被记录，另一个增加就丢失了。

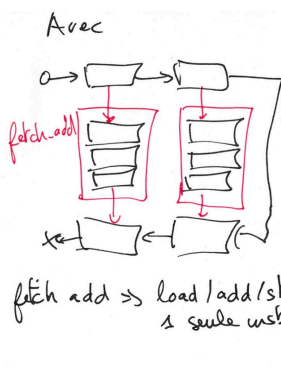
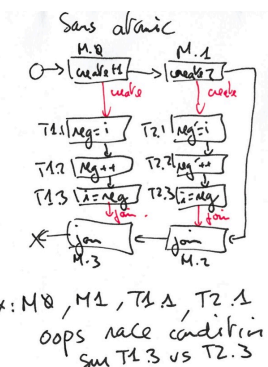
有同步机制的情况 ("Avec") :

当使用原子操作 (如fetch_add) 时，读取、增加和存储余额的操作是作为一个不可分割的单元执行的。这意味着即使多个线程同时执行这些操作，每一次增加都会被正确地记录，因此最终余额将是正确的

ça marche pas bien. On manque d'ordre/contraintes sur les actions des threads. Du coup le résultat est mal déterminé.

Ce dessin explique le problème. On doit décomposer l'incrément du solde en 3 instructions : load, add, store en pratique. On peut donc exhiber des traces problématiques, dès que les deux thread commencent à travailler en même temps (i.e. se passent la main alors qu'ils sont dans le traitement).

On a donc assez probablement un solde inférieur à ce qu'il devrait être



Q6

Si le nombre de pièces d'or du jackpot est faible (disons 100), on n'observera pas de problème en général sur cet exemple,

多线程环境下，线程切换（commutation）可能发生在执行输入/输出操作、显式的sleep或yield调用、或者当线程用尽了它的时间片（quantum exhaustion）时。

在本例中，如果每个线程要增加的金币数量只有100，那么每个线程的工作量非常小，可能在单个时间片内就完成了它的任务，因此线程间几乎没有切换的机会。

即使是在多核心处理器上，由于执行时间短，线程之间交错执行（可能导致数据竞态的情况）的机会也减少了。

C'est lié aux causes possibles de commutation : E/S, sleep ou yield explicite, synchro, épuisement de quantum.

Ici c'est l'épuisement de quantum la source de commutation intempestive principale, donc si compter 100 pièces prend moins d'un quantum, vu qu'on ne fait que ça, peu de chance de commuter.

Le programme reste faux (!)

On refuse dans l'UE e.g. les synchro avec des sleep pour attendre les autres. Ou de compter sur des effets dûs au quantum/hypothèses en général sur le scheduler.

Ici en multi core, on peut encore observer des effets mauvais, malgré le temps court d'exécution.

Q7

Comment utiliser un atomic pour corriger ces problèmes ? Expliquez l'effet au niveau des entrelacements possibles.

关键点是solde变量现在被声明为std::atomic<int>类型，这意味着对solde的所有操作都是原子的，即它们不会被线程调度中断。这是因为atomic类型提供了一种机制，确保一个线程在修改变量的值时，其他线程不能同时修改相同的变量。

```
#include <thread>
#include <iostream>
#include <atomic>
using namespace std;

class Compte {
    // 使用原子类型atomic<int>来声明余额，以确保线程安全
    atomic<int> solde;
public:
    Compte(int solde=0):solde(solde) {}
    void creditor (size_t val) { solde+=val ;}
    int getSolde() const {return solde;}
};
```

Si créditer utilise bien += pour ajouter au solde, le reste du code n'est pas modifié. Utiliser solde = solde + val; est incorrect par contre.

Le dessin du corrigé de la question précédente (à droite) explique le résultat. Les trois actions load/ad- d/store sont groupées en une seule atomiquement (au niveau hardware).

Sans avoir ajouté d'ordre (synchro) entre les instructions des deux thread, on parvient quand même à écarter les exécutions problématiques du cas sans protection.

添加了一个新的操作bool debiter(int val)到Compte类中。这个操作的目的是在确保账户余额充足的情况下扣除一定的金额，并返回一个布尔值表示扣款是否成功

1.3 Section Critique

```
class Compte {
    // ...
    atomic<int> solde; // 使用原子变量来存储余额
public:
    // ...
    bool debiter(unsigned int val) {
        bool doit = (solde >= val);
        if (doit) {
            solde -= val; // 这里需要注意，即使solde是原子类型，这个复合操作不是原子的
        }
        return doit;
    }
    // ...
};

void losepot(Compte & c) {
    for (int i=0; i < JP / 10; i++) {
        c.debiter(10); // 尝试从账户中扣除10
    }
}
```

Q8

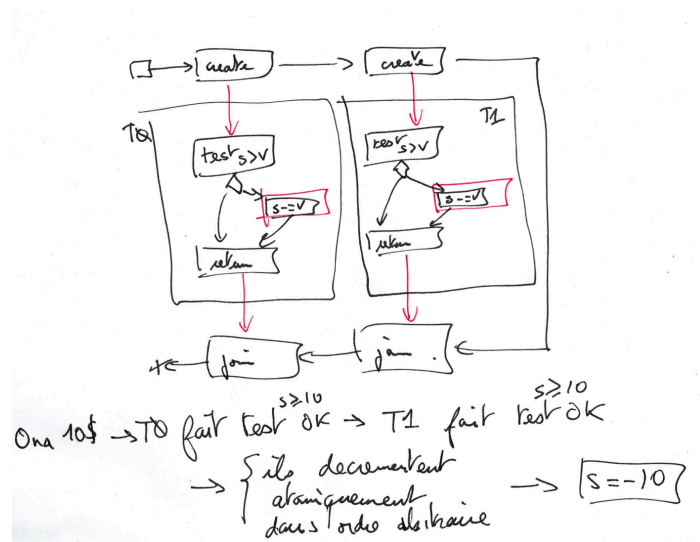
On reprend l'exemple du Jackpot, mais cette fois-ci en débit, le LosePot retire 10000 par paquets de 10. On lance N thread qui exécutent cette fonction ; le compte peut-il tomber en négatif ? Quelle garantie fournit atomic ici ?

Oui, on tombe joyeusement en négatif. On peut atteindre au plus $-10 \times$

NB_THREAD

Le test et sa mise à jour ne sont pas atomiques.

atomic fournit toujours une garantie sur les exécutions de solde = val, donc on aura une valeur cohérente dans solde à la n avec le nombre de fois où débiter à rendu true



Q9. Introduire un mutex dans la classe Compte pour sécuriser son utilisation dans un contexte Multi-Thread.

```
class Compte {
    mutable mutex m; // 可变互斥锁, 即使在const成员函数中也可以被锁定和解锁
    int solde; // 账户余额

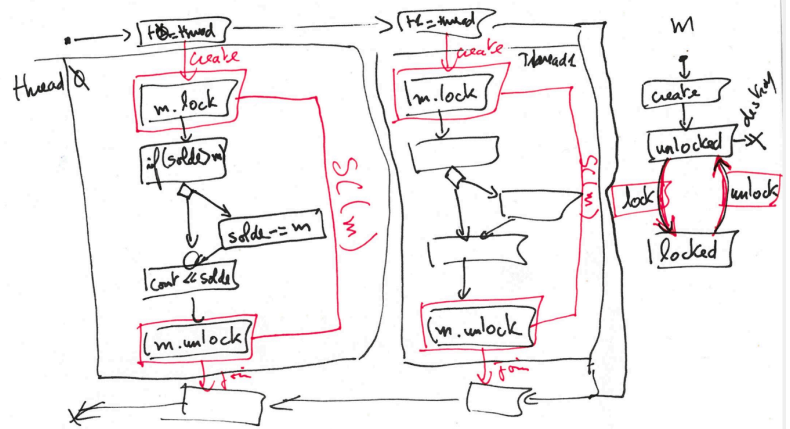
public:
    // 构造函数: 如果没有提供初始余额, 则余额为0
    Compte(unsigned int solde=0):solde(solde) {}

    // 存款方法: 存入一定金额
    void creditor(unsigned int val) {
        m.lock(); // 加锁, 防止其他线程同时修改余额
        solde += val; // 余额增加val
        m.unlock(); // 解锁
    }

    // 取款方法: 尝试取出一定金额
    bool debiter(unsigned int val) {
        m.lock(); // 加锁
        bool doit = solde >= val; // 检查余额是否足够
        if (doit) { // 如果足够, 则进行扣款
            solde -= val; // 余额减少val
        }
        m.unlock(); // 解锁
        return doit; // 返回操作是否成功
    }

    // 查询余额: 返回当前余额
    int getSolde() const {
        m.lock(); // 加锁
        auto ret = solde; // 获取当前余额
        m.unlock(); // 解锁
        return ret; // 返回余额
    }
};
```

Sur le dessin, on a les deux thread de d'habitude, mais aussi le lock, qui a son propre état interne.
Quand on franchit l'action lock/unlock dans le thread, simultanément il faut franchir l'arc rouge du mutex.
NB: c'est le même thread qui doit faire lock et unlock avec des mutex. cf. Sémaphores pour des synchros où l'on débloquent l'autre thread.
On voit aussi ici qu'on doit déclarer le mutex comme étant mutable, un nouveau mot clé pour contourner la nature const des accesseurs en lecture.



→ état du mutex séparé → partage -

Q10

Utiliser un `unique_lock` plutôt que l'API lock/unlock. Comparer les syntaxes.

Donc la création du `unique_lock` va faire directement un lock sur le mutex (sauf si on demande `std::defer`). Sa destruction (i.e. son destructeur) fait le unlock automatiquement.

Il protège donc le bloc dans lequel il est déclaré : comme c'est une variable ordinaire, il est stack alloc, et sera détruit en sortant du bloc.

Il évite donc d'oublier les unlock, particulièrement quand la structure de contrôle est complexe (e.g. plusieurs "return", exceptions, ...).

`unique_lock` est un objet, il sera créé lors de la création du mutex, et sera détruit lors de la destruction du mutex.

Quand on utilise un mutex pour protéger une section critique, on utilise généralement `lock()` et `unlock()`. Mais si on utilise `unique_lock`, on n'a pas besoin de faire ces deux appels. `unique_lock` est un objet, il sera créé lors de la création du mutex, et sera détruit lors de la destruction du mutex.

Si on utilise `unique_lock`, on n'a pas besoin de faire ces deux appels. `unique_lock` est un objet, il sera créé lors de la création du mutex, et sera détruit lors de la destruction du mutex.

Le texte mentionne également un cas particulier : si on utilise `unique_lock` avec `std::defer`, on n'a pas besoin de faire ces deux appels. `unique_lock` est un objet, il sera créé lors de la création du mutex, et sera détruit lors de la destruction du mutex.

Enfin, le texte mentionne également un cas particulier : si on utilise `unique_lock` avec `std::defer`, on n'a pas besoin de faire ces deux appels. `unique_lock` est un objet, il sera créé lors de la création du mutex, et sera détruit lors de la destruction du mutex.

Enfin, le texte mentionne également un cas particulier : si on utilise `unique_lock` avec `std::defer`, on n'a pas besoin de faire ces deux appels. `unique_lock` est un objet, il sera créé lors de la création du mutex, et sera détruit lors de la destruction du mutex.

```
class Compte {
    // mutable permet de modifier le mutex même dans une méthode const
    mutable mutex m; // 可变互斥锁, 即使在const方法中也可以被修改
    int solde; // 账户余额

public:
    Compte(int solde=0):solde(solde) {}

    void creditor(unsigned int val) {
        unique_lock<mutex> g(m); // 使用unique_lock自动管理锁的生命周期
        solde+=val;
    }

    bool debiter(unsigned int val) {
        unique_lock<mutex> g(m); // 自动加锁
        bool doit = solde >= val;
        if (doit) {
            solde -= val;
        }
        // 自动解锁, 当g离开作用域时
        return doit;
    }

    bool debiter2(unsigned int val) {
        unique_lock<mutex> g(m); // 自动加锁
        if (solde >= val) {
            solde -= val;
            return true;
        } else {
            return false;
        }
        // 自动解锁
    }

    bool debiter3(unsigned int val) {
        m.lock(); // 手动加锁
        if (solde >= val) {
            solde -= val;
            m.unlock(); // 手动解锁
            return true;
        } else {
            m.unlock(); // 记得解锁
            return false;
        }
        // 这里没有自动解锁的机制
    }

    int getSolde() const {
        unique_lock<mutex> g(m); // 使用unique_lock自动加锁
        return solde;
        // 自动解锁
    }
};
```

Q11

Dans les versions avec un mutex, qu'apporte l'utilisation d'un atomic pour l'attribut solde ? Si l'on n'utilise pas atomic est-ce nécessaire de protéger la méthode getSolde avec le mutex ?

Si le mutex protège déjà les accès au solde, atomic semble ne pas apporter grand chose. La seule chose qu'il garantit c'est les load/store atomic à ce stade : le mutex protège déjà.

Si getSolde n'est pas protégé, a priori, ça ne devrait rien changer : On fait return solde, on lira la valeur avant ou après une écriture (dans tous les cas ce sera OK a priori).

En réalité on est déjà dans du Undened Behavior ! Il est interdit de lire et écrire à la même adresse simultanément !

C'est ok si : IF you're reading/writing 4-byte value AND it is DWORD-aligned in memory AND you're running on the I32 architecture, THEN reads and writes are atomic.

En général c'est du UB en C++, atomic fera ce qu'il faut faire sur votre archi matérielle.

Si le solde était un long double e.g. qui fasse plus qu'un mot mémoire, on voit mieux les problèmes ! Donc c'est une faute de lire pendant que quelqu'un y écrit (on peut lire la moitié de l'écriture de l'autre).

Il faudrait donc remettre de l'atomic ! Mais vu qu'on a construit un mutex par Compte, autant aller au bout de l'approche, la version mutex n'a pas besoin d'utiliser atomic, mais doit bien protéger tous ses accès avec le mutex, lecture et écriture.

Cela nous donne une forme générale pour une classe MT-safe sans trop rééchir, similaire à ce qu'on a en annotant une classe Java par synchronized sur chaque méthode.

Q12

La classe Compte en l'état ne dispose pas de constructeur par copie : mutex n'est pas copiable, ce qui rend la version générée par le compilateur de ce constructeur par copie non disponible. Comment écrire ce constructeur ?

NB: Sans constructeur par copie, on ne peut par exemple pas insérer un Compte dans un std::vector.

类Compte在当前状态下没有复制构造函数，因为mutex不可复制，这使得编译器生成的默认复制构造函数不可用。如果不使用复制构造函数，应该如何编写这个构造函数？

注意：没有复制构造函数，我们例如不能将Compte插入到std::vector中。

Donc mutex n'est pas copiable, ce qui rend la version générée par le compilo du ctor par copie non disponible. Il faut locker l'objet source pour faire la copie.

transferts.cpp

```
// NB : vector exige Copyable, mais mutex ne l'est pas...
Compte(const Compte & other) {
    other.lock();
    solde = other.solde;
    other.unlock();
}
```

TME 4

Mutex et section critique

atomic, mutex

• sections critique, interblocage

```
class Compte {
    mutable mutex m; // 可变互斥锁, 允许在const成员函数中被锁定
    int solde;        // 账户余额

public:
    // 构造函数
    Compte(int solde=0): solde(solde) {}

    // 存款函数
    void creditor(unsigned int val) {
        unique_lock<mutex> g(m); // 使用unique_lock自动管理锁
        solde += val;            // 增加账户余额
    }

    // 取款函数
    bool debiter(unsigned int val) {
        unique_lock<mutex> g(m); // 自动加锁
        bool doit = solde >= val; // 检查余额是否充足
        if (doit) {
            solde -= val;        // 执行取款操作
        }
        return doit;            // 返回操作是否成功
    }

    // 查询余额函数
    int getSolde() const {
        unique_lock<mutex> g(m); // 使用unique_lock自动加锁
        return solde;            // 返回当前余额
    }

    // 注意: std::vector 需要可复制的类型, 但 mutex 不可复制...
    // 复制构造函数, 尝试从另一个Compte对象复制状态
    Compte(const Compte &other) {
        other.m.lock();          // 尝试锁定源对象的互斥锁 (但这在const对象上是非法的)
        solde = other.solde;     // 复制余额
        other.m.unlock();        // 解锁源对象的互斥锁
    }
};
```

```
class Banque {
    // 定义一个类型为 Compte 的 vector, 命名为 comptes_t
    typedef vector<Compte> comptes_t;
    comptes_t comptes; // 银行账户的容器

public:
    // Banque 的构造函数, 初始化一定数量的账户和每个账户的初始余额
    Banque(size_t ncomptes, size_t solde) : comptes(ncomptes, Compte(solde)) {}

    // 转账函数, 尝试从一个账户转账到另一个账户
    void transfer(size_t deb, size_t cred, unsigned int val) {
        Compte &debiteur = comptes[deb]; // 获取转出账户的引用
        Compte &crediteur = comptes[cred]; // 获取转入账户的引用
        if (debiteur.debiter(val)) { // 如果转出成功
            crediteur.crediteur(val); // 执行转入操作
        }
    }

    // 获取银行账户总数的函数
    size_t size() const { return comptes.size(); }
};
```

```
// 线程执行的函数
void transferJob(int index, Banque &banque) {
    std::cout << "started " << index << std::endl;
    for (int i = 0; i < 10000; ++i) {
        int debite = rand() % banque.size(); // 随机选择一个转出账户
        int credite = rand() % banque.size(); // 随机选择一个转入账户
        int val = rand() % 70 + 30; // 随机生成转账金额
        banque.transfer(debite, credite, val); // 执行转账
        //std::this_thread::sleep_for(std::chrono::milliseconds(rand()%10)); // 随机暂停
    }
    std::cout << "finished " << index << std::endl;
}
```

```
// 主函数
int main() {
    using namespace pr;
    const int N = 10; // 线程数
    ::srand(::time(nullptr)); // 随机数种子
    vector<thread> threads; // 线程容器
    threads.reserve(N); // 为线程容器预留空间

    Banque b(200, 100); // 创建一个银行对象, 含有200个账户, 每个账户初始余额为100

    for (int i=0; i < N; ++i) {
        // 创建线程, 传递 transferJob 函数和银行对象的引用
        threads.emplace_back(transferJob, i, std::ref(b));
        std::cout << "created " << i << std::endl;
    }
    // 变量i用于给每个线程一个唯一的索引或者ID, 它被传递到 transferJob 函数中, 用于区分不同的线程
    for (auto &th : threads) {
        th.join(); // 等待所有线程完成
        std::cout << "joined " << i << std::endl;
    }
    return 0;
}
```

Q2

Le comportement est-il correct (pas de data race) avec les protections actuelles sur le Compte ?

A priori oui, notre classe Compte est MT-safe, on peut s'en servir dans n'importe quel contexte MT sans risquer de fautes. D'où l'intérêt des classes de librairie MT safe

Q3

La solution avec les trois méthodes permet de respecter le contrat d'un Lockable (mutex), i.e. on peut passer des comptes à std::lock la version multiple. Le corrigé adopte cette approche, donc on fera `compte.lock()` plutôt que `compte.getLock().lock()`. Ajouts dans la classe Compte :

```
// 访问互斥锁的方法, 供银行使用
void lock() const {
    m.lock();
}

void unlock() const {
    m.unlock();
}

// 为了满足 Lockable 协议要求而实现的方法
bool try_lock() const {
    return m.try_lock();
}
```

```
#include <thread>
#include <mutex>
#include <atomic>
#include <iostream>
#include <random>
using namespace std;
// 账户类
class Compte {
    mutable recursive_mutex m; // 递归互斥锁, 允许同一个线程多次加锁
    int solde; // 账户余额

public:
    Compte(int solde=0): solde(solde) {}

    // 存款方法
    void creditor(unsigned int val) {
        lock_guard<recursive_mutex> g(m); // 加锁
        solde += val; // 增加余额
    }

    // 取款方法
    bool debiter(unsigned int val) {
        lock_guard<recursive_mutex> g(m); // 加锁
        bool doit = solde >= val; // 检查余额是否充足
        if (doit) {
            solde -= val; // 执行扣款
        }
        return doit; // 返回是否扣款成功
    }

    // 查询余额方法
    int getSolde() const {
        lock_guard<recursive_mutex> g(m); // 加锁
        return solde; // 返回余额
    }

    // 锁定账户的方法, 供银行使用
    void lock() const {
        m.lock();
    }

    // 解锁账户的方法
    void unlock() const {
        m.unlock();
    }
};

// 定义一个名为 comptableJob 的函数, 该函数接收一个常量引用的 Banque 类对象和一个迭代次数。
void comptableJob (const Banque & banque, int iterations) {
    // 使用 for 循环进行迭代
    for (int iter = 0 ; iter < iterations ; iter++) {
        // 如果 Banque 类的 comptabiliser 方法返回 false, 表示账目出现问题, 打印错误信息
        if (! banque.comptabiliser()) {
            cout << "Comptable faché !" << endl;
        }
        // 否则打印账目已经完成的信息
        cout << "Bilan " << iter << " fini." <<endl;
        // 每次迭代结束后暂停20毫秒
        std::this_thread::sleep_for (std::chrono::milliseconds(20));
    }
}

// 定义一个名为 transfertJob 的函数, 该函数接收一个线程索引和一个常量引用的 Banque 类对象。
void transfertJob (int index, Banque & banque) {
    // 打印线程开始的信息
    std::cout << "started " << index << endl;
    // 使用 for 循环模拟银行转账操作
    for (int i = 0 ; i < 100000; i++) {
        // 生成随机的借记和贷记账户索引
        int debite = rand() % banque.size();
        int credite = rand() % banque.size();
        // 生成随机的转账金额
        int val = rand() % 70 + 30;
        // 进行转账
        banque.transfert(debite,credite,val);
        // 每次转账后随机暂停0到9毫秒
        //std::this_thread::sleep_for(std::chrono::milliseconds(rand()%10));
    }
    // 打印线程结束的信息
    std::cout << "finished " << index << endl;
}

// 银行类
class Banque {
    typedef vector<Compte> Comptes; // 账户的容器
    Comptes comptes;
    const int initial; // 每个账户的初始余额

public:
    // 银行构造函数
    Banque(size_t ncomptes, size_t solde) : comptes(ncomptes, Compte(solde),initial(solde)) {}
    // 转账方法, 实现了锁定顺序以避免死锁
    void transfer(size_t deb, size_t cred, unsigned int val) {
        Compte& debiteur = comptes[deb]; // 转出账户
        Compte& creditur = comptes[cred]; // 转入账户
        if (deb < cred) { // 确保锁定顺序
            debiteur.lock(); // 先锁定索引较小的账户
            creditur.lock();
        } else {
            creditur.lock(); // 先锁定索引较小的账户
            debiteur.lock();
        }
        if (debiteur.debiter(val)) { // 执行转账
            creditur.crediter(val);
        }
        debiteur.unlock(); // 解锁
        creditur.unlock();
    }

    // 获取银行账户总数的方法
    size_t size() const {
        return comptes.size();
    }

    // 会计方法, 对所有账户余额进行计算, 确保总额不变
    bool comptabiliser() const {
        int attendu = initial * comptes.size(); // 预期总额
        int bilan = 0; // 实际总额
        int id = 0;
        for (const auto &compte : comptes) {
            compte.lock(); // 锁定账户
            if (compte.getSolde() < 0) { // 检查是否有负余额
                cout << "Compte " << id << " en négatif : " << compte.getSolde() << endl;
            }
            bilan += compte.getSolde(); // 累加余额
            id++;
        }
        for (const auto &compte : comptes) {
            compte.unlock(); // 解锁账户
        }
        if (bilan != attendu) {
            cout << "Bilan comptable faux: attendu " << attendu << " obtenu : " << bilan << endl;
        }
        return bilan == attendu;
    }
};

// 定义一个名为 createAndWait 的函数, 该函数接收一个表示线程数的整数 N。
int createAndWait (int N) {
    // 创建一个 thread 类型的向量 threads 来存储线程
    vector<thread> threads;
    // 预先分配线程向量的空间
    threads.reserve(N);

    // 创建一个 Banque 类对象 b, 初始化为200个账户和每个账户100单位的金额
    Banque b(200,100);

    // 使用 for 循环创建 N-2 个转账线程
    for (int i=0; i < N - 2; i++) {
        // 创建线程并加入到 threads 向量中
        threads.emplace_back(thread(transfertJob,i,std::ref(b)));
        // 打印线程创建的信息
        std::cout << "created " << i << endl;
    }
    // 创建 2 个 comptableJob 线程
    for (int i = N - 2 ; i < N ; i++) {
        // 创建线程并加入到 threads 向量中
        threads.emplace_back(thread(comptableJob,std::cref(b),10));
        // 打印线程创建的信息
        std::cout << "created " << i << endl;
    }
    // 使用 for 循环等待所有线程结束
    for (int i=0; i < N ; i++) {
        // 等待 threads 向量中的第 i 个线程结束
        threads[i].join();
        // 打印线程结束的信息
        std::cout << "joined " << i << endl;
    }
    // 函数返回 0 表示成功
    return 0;
}

// 程序的入口点 main 函数
int main16 () {
    // 使用 srand 函数设置随机数种子
    ::srand(::time(nullptr));
    // 调用 createAndWait 函数并返回其结果
    return exBanque::createAndWait(10);
}
```